

GCC 128 — INTELIGÊNCIA ARTIFICIAL

Classificação com KNN

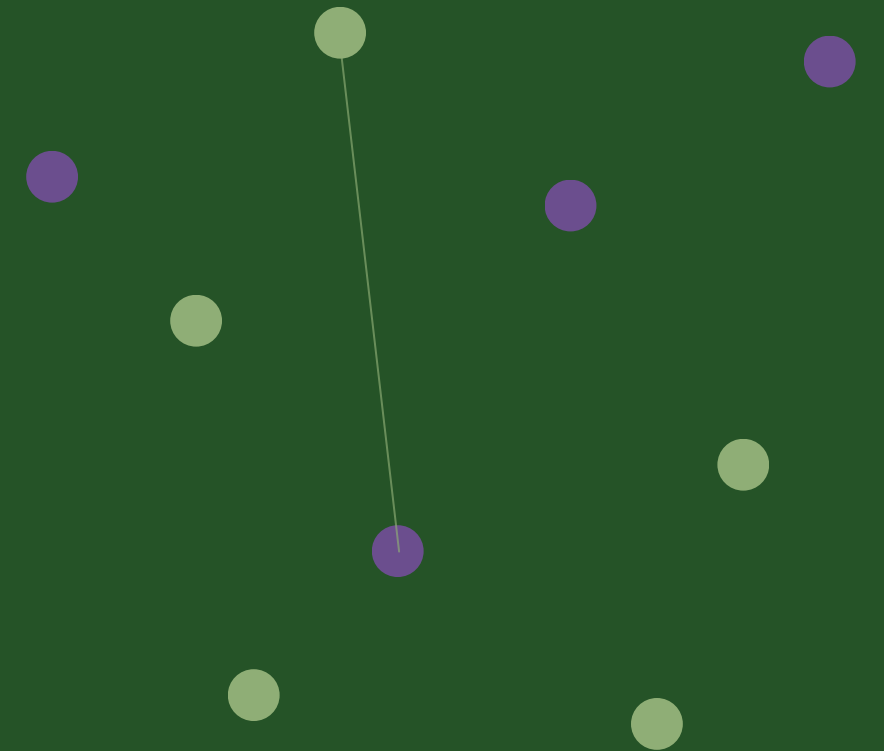
K-Nearest Neighbors implementado do zero e comparado com o Scikit-learn, aplicado à base de dados Iris.

PROFESSOR

Ahmed Ali Abdalla Esmin

ALUNO

Tobias Maugus Bueno Cougo



Objetivo da atividade

Fixar os conceitos de classificação supervisionada implementando o algoritmo KNN e comparando-o com uma solução consolidada de mercado.



Implementar do zero

Desenvolver o KNN (hardcore) em Python/NumPy, sem bibliotecas prontas do algoritmo.



Testar $k = \{1, 3, 5, 7\}$

Medir a taxa de reconhecimento (acurácia) do classificador na base Iris para cada k .



Avaliar formalmente

Gerar matriz de confusão e métricas: precisão, revocação e acurácia.



Comparar com o Scikit-learn

Confrontar a implementação manual com o KNeighborsClassifier já pronto.

Como o KNN classifica um ponto novo

1

Calcula a distância

Mede a distância Euclidiana do novo ponto até todos os pontos de treino.

2

Seleciona os k vizinhos

Ordena as distâncias e escolhe os k pontos de treino mais próximos.

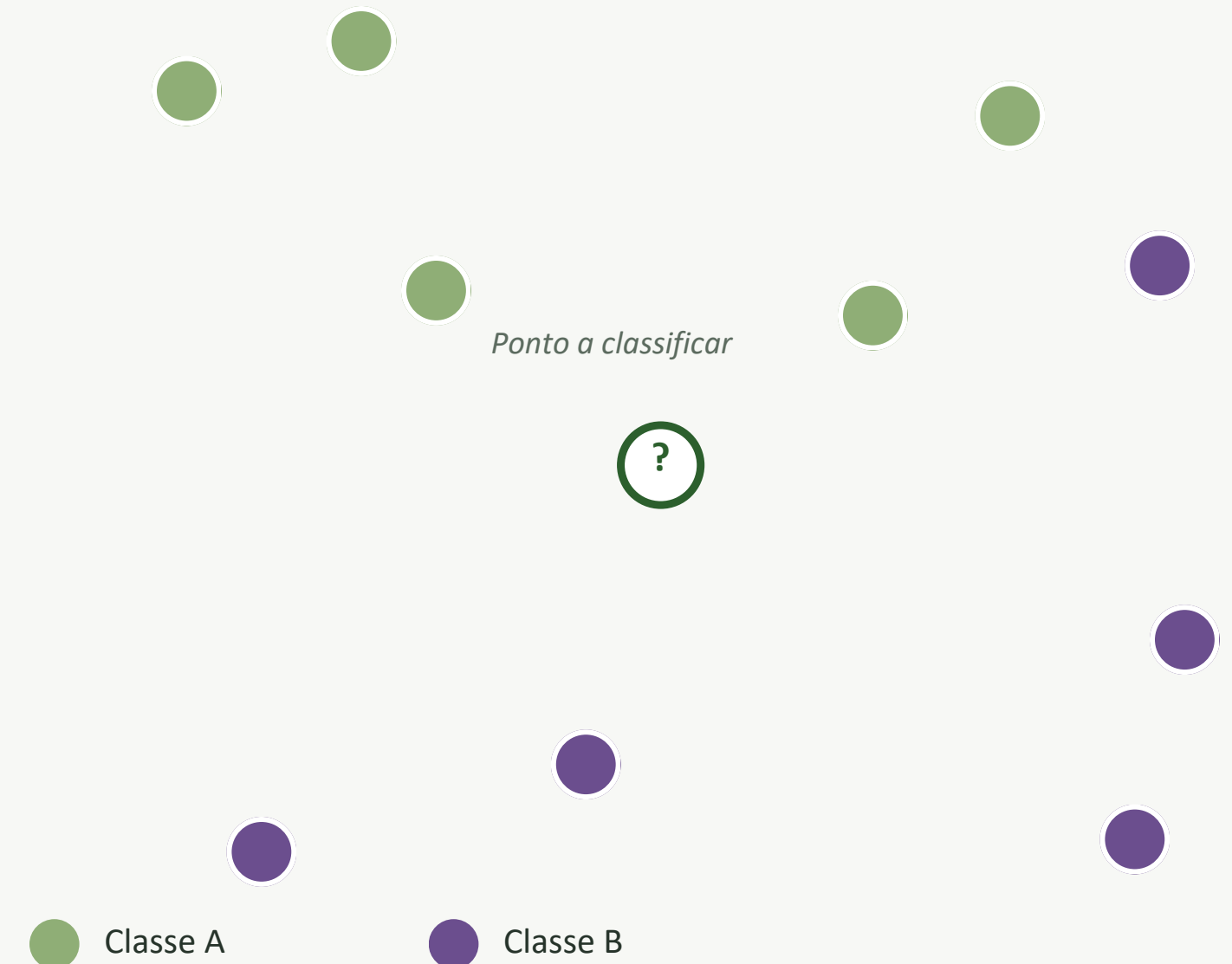
3

Vota pela maioria

A classe atribuída é a mais frequente entre esses k vizinhos.

k é um hiperparâmetro: poucos vizinhos $\rightarrow$ sensível a ruído; muitos vizinhos $\rightarrow$ fronteira mais suave.

$k = 3 \rightarrow 2$ vizinhos da Classe A, 1 da Classe B $\rightarrow$ previsão: Classe A



A base de dados Iris

Clássica em aprendizado de máquina: mede sépalas e pétalas de flores para distinguir três espécies.

150

amostras

3

espécies (classes)

4

atributos numéricos

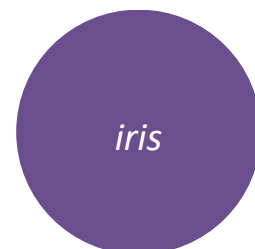
70/30

divisão treino/teste

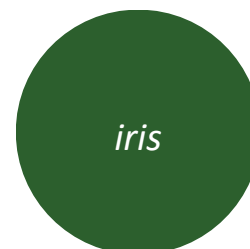
Espécies classificadas



Setosa



Versicolor



Virginica

Atributos utilizados

- Comprimento da sépala (cm)
- Largura da sépala (cm)
- Comprimento da pétala (cm)
- Largura da pétala (cm)

Divisão estratificada por classe, com random_state = 42 para reprodutibilidade.

Implementação hardcore (do zero)

Classe Python própria, usando apenas NumPy — sem qualquer biblioteca com o KNN pronto.

1**fit(X, y)**

Apenas armazena os dados de treino — o KNN é um algoritmo “lazy” (preguiçoso).

**2****distância**

Para cada amostra de teste, calcula a distância Euclidiana até todos os pontos de treino.

**3****predict(X)**

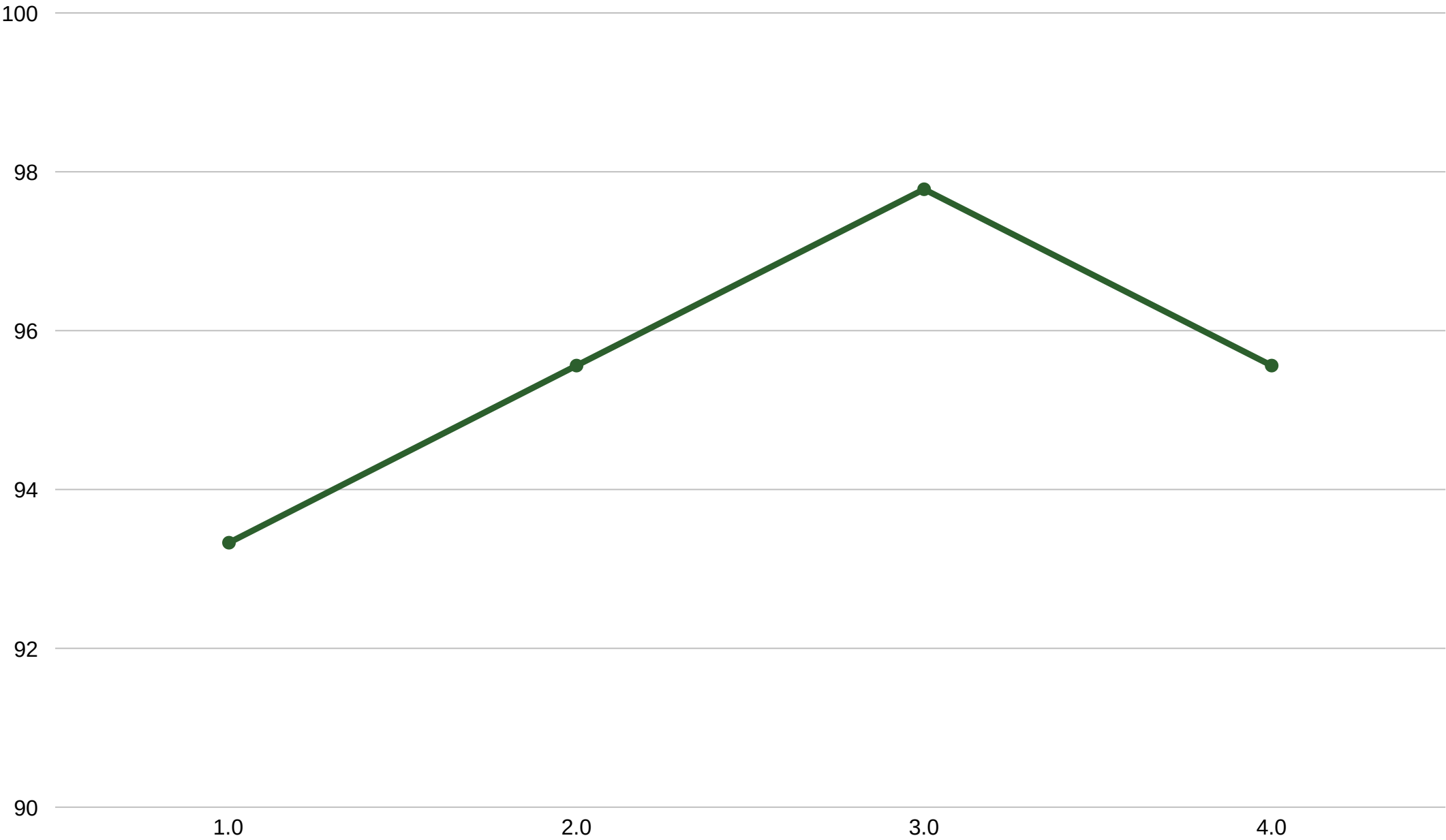
Ordena as distâncias, pega os k menores e retorna a classe mais votada entre eles.

```
distancias = [np.sqrt(np.sum((x - x_treino) ** 2)) for x_treino in self.X_train]
k_indices  = np.argsort(distancias)[:self.k]
k_rotulos  = self.y_train[k_indices]
valores, contagens = np.unique(k_rotulos, return_counts=True)
return valores[np.argmax(contagens)] # classe mais votada
```


Taxa de reconhecimento para $k = \{1, 3, 5, 7\}$

As duas implementações produzem exatamente as mesmas acurácias em todos os valores de k testados.

Curva única: hardcore e Scikit-learn coincidem em todos os pontos.



MELHOR RESULTADO

k = 5
97,78%

de acurácia — o mesmo valor obtido tanto pela implementação hardcore quanto pelo Scikit-learn.

k = 1 teve o pior resultado (93,33%) — mais sensível a ruído nos vizinhos mais próximos.

Matriz de confusão e métricas (k = 5)

Avaliação da implementação hardcore no melhor valor de k, no conjunto de teste (45 amostras).

Real \ Previsto	Setosa	Versicolor	Virginica
Setosa	15	0	0
Versicolor	0	15	0
Virginica	0	1	14

Apenas 1 amostra de Virginica foi confundida com Versicolor — o restante foi classificado corretamente.

97,78%

Acurácia

97,92%

Precisão (macro)

97,78%

Revocação (macro)

Hardcore vs. Scikit-learn

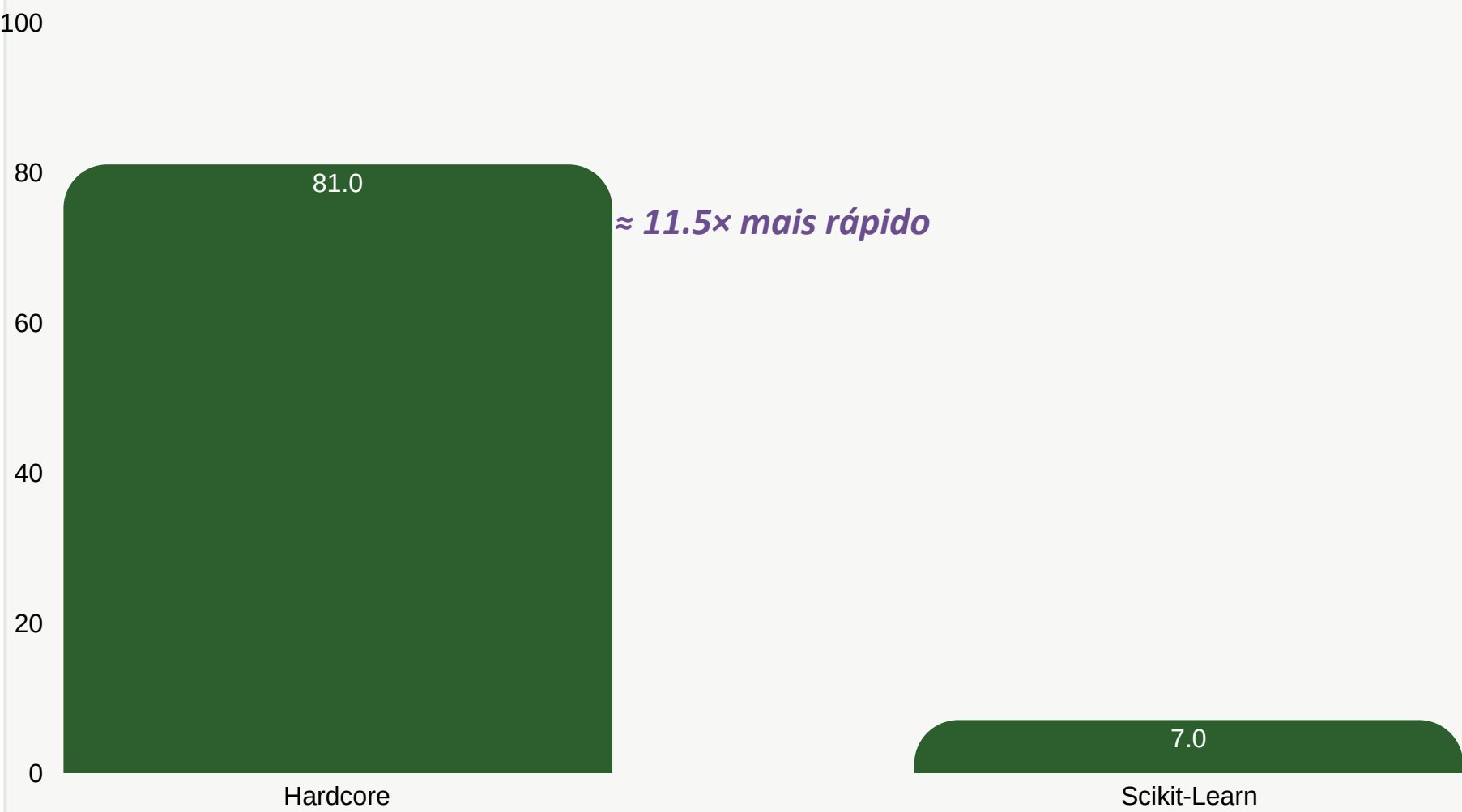
Mesma qualidade de classificação — desempenho computacional bem diferente.

Qualidade da classificação

Métrica	Hardcore	Sklearn
Acurácia	0,9778	0,9778
Precisão	0,9792	0,9792
Revocação	0,9778	0,9778

Resultados idênticos em todas as métricas

Tempo médio de predição (k = 1..7) em milissegundos



CONCLUSÃO

O que aprendemos



Entendimento profundo do algoritmo

Implementar o KNN do zero deixou claro cada etapa: distância, ordenação e votação — não é uma “caixa-preta”.



Mesma qualidade, contextos diferentes

Hardcore e Scikit-learn chegam às mesmas métricas; a biblioteca pronta vence em velocidade e escalabilidade.



$k = 5$ foi o ponto ótimo

Equilibra sensibilidade a ruído (k baixo) e suavização excessiva da fronteira (k alto) para a base Iris.

Obrigado!